

DSA Project - Plag Checker

Ishita - 23b0921

Anushka Singhal - 23b0928

August 2024

Contents

1	Phase 1 : Barebones Checking of Two Submissions	2
1.1	Algorithm and Method	2
1.2	Flagging conditions	2
1.3	Data Structures used and overall flow of code	2
1.4	Time Complexity and Space Complexity	2
2	Phase 2 : A Full-Fledged Plagiarism Checker	3
2.1	Algorithms and Function Signatures	3
2.2	Complexity Analysis	3
2.3	Threading and Concurrency	3
2.4	File Selection for Matching	3
2.5	Helpers and Overall Flow	3

1 Phase 1 : Barebones Checking of Two Submissions

1.1 Algorithm and Method

1.1.1 Exact Matches

For this part, **Polynomial Rolling Hashing technique** is used to identify identical sequences (tokens) of length more than 10 between the two submissions. Initially, hashes are computed for all tokens of length 10 in the first submission. These hashes are then compared against the corresponding hashes from the second submission, which are also calculated using a rolling hash method for computational efficiency.

When a hash from the first submission matches a hash from the second, the corresponding positions in the first file are marked as part of an exact match. As we take hashes of length 10, exact patterns of less than 10 will not be considered and anything above 10 will be considered due to marking of positions and counting them. To avoid overlapping matches, this process is repeated with the roles of the two submissions reversed (i.e., computing hashes for the second submission and comparing them to the first). Finally, the minimum count of marked positions between the two files is taken, representing the length of the exact pattern matches.

The time complexity for this section is $O(n + m)$, where n and m are the sizes of the two submissions.

1.1.2 Approximate matching

To identify near-identical patterns between two submissions **Dynamic Programming** is used.

The function `std::vector<std::vector<int>>> edit_distance` computes the edit distance matrix, where each cell represents the cost of transforming one subsequence into another through operations such as insertion, deletion, or replacement. Using this matrix, the algorithm searches for potential subsequences by starting with the longest possible matches and gradually reducing their length. A subsequence is classified as an approximate match if its edit distance falls within a specified threshold (20% of its length), indicating minimal differences. The algorithm identifies the longest such match and records its starting indices in both submissions.

1.2 Flagging conditions

The submissions are flagged as potentially plagiarized if either:

- The exact match length is greater than or equal to 50% of the smaller submission's size.
- The longest approximate match length is greater than or equal to 70% of the smaller submission's size.

These thresholds ensure that significant overlap in either exact or near-exact sequences triggers plagiarism detection, balancing sensitivity and accuracy.

1.3 Data Structures used and overall flow of code

The flow starts with precomputing hash powers and calling `find_exact_matches`, which relies on helper functions such as `hash_function` for initial hash computation, `rolling_hash` for efficiently updating hash values, and `calculate_hashes` to compute hashes for all tokens in a submission. The function matches hashes between submissions and marks overlapping positions.

Next, `find_approximate_matches` is called, which uses `edit_distance` to build a matrix for determining near-identical patterns. The results of exact and approximate matches are compared against predefined thresholds according to the flagging conditions. In data structures mainly vectors are used, in addition to multiset which was used to store and efficiently manage the hashes of tokens from the submissions, allowing for fast lookup and deletion of matching hashes to detect exact matches while avoiding overlapping matches.

1.4 Time Complexity and Space Complexity

- The overall Time Complexity is $O(n^3)$
This is because the subsequence search in the approximate matching step involves three nested loops for submissions of size n and m .
- The overall Space Complexity is $O(n * m)$
The space complexity is dominated by the 2D DP matrix used for edit distance calculation.

2 Phase 2 : A Full-Fledged Plagiarism Checker

2.1 Algorithms and Function Signatures

- **Long Pattern Matches:** Hashes of tokens (length `TOKEN_LENGTH`) are calculated using rolling hash, same as phase 1. Matches are identified by checking token hashes between submissions with length greater than `EXACT_PATTERN_MATCH`
- **Patchwork Plagiarism:** The number of pattern matches of length `TOKEN_LENGTH` is calculated between the current file and all previously processed files, and the total count of pattern matches is incremented accordingly.

2.2 Complexity Analysis

Let m be the number of files and n the number of tokens per file. The complexity for key operations is as follows:

1. **Long Pattern Matching:**
 - *Time:* $\mathcal{O}(n)$ for hash calculation and $\mathcal{O}(n)$ for matching per file.
 - *Space:* $\mathcal{O}(n)$.
2. **Patchwork Detection:**
 - *Time:* $\mathcal{O}(m \cdot n)$ for all files.
 - *Space:* $\mathcal{O}(n \cdot m)$.

This analysis demonstrates the system’s scalability with respect to the number of tokens and files, optimizing for both time and space efficiency.

2.3 Threading and Concurrency

Threading and concurrency are implemented to ensure non-blocking `add_submission()` calls on the main thread. Worker threads operate in `worker_function()` by processing pending submissions from a shared queue. When a submission is added through `add_submission()`, it is queued, and the `std::condition_variable` notifies worker threads. These threads utilize `std::mutex` to safely access and process submissions concurrently, maintaining synchronization and ensuring that the main thread remains responsive while submissions are handled in the background.

2.4 File Selection for Matching

During matching, each file is compared against all previously processed files. If plagiarism is detected and the time difference between submissions is less than one second, both files are flagged. Otherwise, only the newer file is flagged as plagiarized.

2.5 Helpers and Overall Flow

- `calculate_hashes_for_token_vector()`: Computes rolling hashes.
- `worker_function()`: Concurrently processes submissions.
- `tokens`: Maps submission IDs to tokenized code.
- `hashes_of_files`: Stores hashes for plagiarism detection.
- `pending_submissions`: Queue for unprocessed submissions.
- `checker_thread`: Thread pool for asynchronous processing.

Flow: The plagiarism checker initializes with `calculate_base_powers`, precomputing hash powers, and starts worker threads in the constructor (`plagiarism_checker_t`). Submissions are added using `add_submission`, which tokenizes code (`tokenizer_t::get_tokens`) and computes hashes (`calculate_hashes_for_token_vector`). Worker threads process submissions via `worker_function`, detecting plagiarism through `detect_plagiarism` and comparing token hashes using `check_files`. Finally, threads and resources are safely terminated in the destructor (`plagiarism_checker_t`).